

O objetivo deste trabalho é desenvolver um sistema de roteamento capaz de encontrar caminhos válidos em uma malha viária real. Serão utilizados os dados da cidade de Ribeirão Preto extraídos do OpenStreetMap (OSM). O mapa foi processado e simplificado em dois arquivos tabulares (.csv) que representam o nosso espaço de estados (um grafo não orientado):

- **nos.csv** (vértices): representa os cruzamentos da cidade. Contém as colunas osmid (identificador único global do nó), latitude (-90 graus, polo sul; +90 graus polo norte) e longitude (-180 graus, oeste; +180 graus, leste).
- **arestas.csv** (conexões): representa os trechos de ruas que ligam os cruzamentos. Contém as colunas origem (osmid de partida), destino (osmid de chegada), distancia (custo do trecho em metros) e nome_ rua.

O seu sistema deve receber um **osmid** de origem A e um **osmid** de destino B e calcular a rota entre eles, com total liberdade para escolher qual algoritmo de busca implementar em cada desafio. A única exigência é que o algoritmo deve ser capaz de encontrar um caminho válido do ponto A ao ponto B, fornecendo o seguinte *output* (exemplo de referência apenas):

```
# -----
# RESULTADO DA BUSCA - DESAFIO 1 (ROTA ÓTIMA)
# -----
# Algoritmo utilizado: [Nome do Algoritmo]
# Origem (osmid): 123456789
# Destino (osmid): 987654321
# -----
# Caminho encontrado (Qtd nós): 45 cruzamentos
# Distância da Rota: 4325.50 metros
# NÓS EXPANDIDOS: 1250
# -----

# -----
# RESULTADO DA BUSCA - DESAFIO 2 (ROTA EXPRESSA)
# -----
# Algoritmo utilizado: [Nome do Algoritmo]
# Origem (osmid): 123456789
# Destino (osmid): 987654321
# -----
# Caminho encontrado (Qtd nós): 42 cruzamentos
# Distância da Rota: 4850.20 metros (+12.13%)
# NÓS EXPANDIDOS: 310 (-75.20%)
# -----
```

Para auxiliar na visualização dos resultados, está sendo fornecido o script **verRota.py** (Python). Esse arquivo recebe a sequência de IDs dos cruzamentos gerada pelo algoritmo do grupo (requer alteração no código fornecido) e plota automaticamente a rota encontrada em um mapa interativo, que pode ser aberto e visualizado diretamente em qualquer navegador web. A utilização desse recurso visual é altamente recomendada, pois permite verificar de forma imediata e prática se o caminho traçado faz sentido na malha viária real, facilitando a depuração do código durante o desenvolvimento e enriquecendo a apresentação final do trabalho.

1. Critérios de Avaliação Base (8,0 Pontos)

Estes pontos avaliam se o grupo cumpriu os requisitos fundamentais de funcionalidade, modularidade, documentação e interface:

- **[2,5 pontos] Correção e Funcionalidade da Busca:** O programa compila/executa sem erros? Consegue carregar a base de dados corretamente? O algoritmo implementado encontra, de fato, um caminho válido entre a origem e o destino propostos, lidando bem com os dados do mapa?
- **[2,0 pontos] Clareza da Implementação e Modularidade:** O código é bem estruturado? As funções (ou predicados, em Prolog) têm responsabilidades únicas e bem definidas? Evitaram "código espaguete" e variáveis com nomes confusos (como x, y, temp)? A estrutura de dados escolhida para a Fronteira (Fila, Pilha, etc) faz sentido?
- **[2,0 pontos] Comentários e Documentação Intracódigo:** Os trechos complexos estão bem explicados? O grupo documentou claramente onde e como o incremento da variável de "nós expandidos" está sendo feito? Se usaram uma heurística, o raciocínio matemático por trás dela está explicado nos comentários?
- **[1,5 ponto] Formatação das Saídas e Apresentação:** O programa imprime os resultados no terminal exatamente no template solicitado (Origem, Destino, Distância, Expansões, Porcentagens)? As mensagens de erro (caso não exista rota) são tratadas de forma amigável?

2. A Competição / Otimização (2,0 Pontos)

Durante a apresentação, o professor fornecerá 3 rotas de teste a serem executadas pelo grupo com seu código. A pontuação será dividida em dois desafios, valendo 1,0 ponto cada, simulando os *trade-offs* de sistemas de roteamento reais.

2.1 Desafio 1: A Rota Ótima (1,0 Ponto)

Desafio 1: A Rota Ótima (1,0 Ponto)

- **Passo 1: O Filtro da Otimalidade (Distância Total):** O algoritmo precisa, obrigatoriamente, encontrar a distância mínima (o caminho ótimo) conhecida para aquela rota. Se o grupo achar um caminho mais longo que o ótimo, ele falha nesta etapa e recebe zero no Desafio 1 daquela rota.
- **Passo 2: O Desempate (Nós Expandidos):** Apenas para os grupos que garantiram a otimalidade, o ranqueamento será feito pela soma de nós expandidos. O grupo mais eficiente da sala leva 1,0 ponto. Os demais recebem a nota proporcional: $Nota = ((Pior\ Expansão) - (Expansão\ do\ Grupo)) / ((Pior\ Expansão) - (Melhor\ Expansão))$

2.2 Desafio 2: A Rota Expressa Subótima (1,0 Ponto)

Neste desafio, o objetivo é velocidade extrema. O sistema pode abrir mão da precisão absoluta em troca de expandir o menor número de nós possível.

- **Passo 1 (O Filtro de Tolerância):** O algoritmo não precisa encontrar a rota perfeita, mas o caminho retornado deve ter uma distância total que seja, no máximo, **15% maior** do que a distância ótima conhecida para a rota. Se o caminho encontrado ultrapassar essa margem de erro, o grupo recebe zero no Desafio 2 daquela rota.
- **Passo 2 (O Desempate da Eficiência Extrema):** Entre os grupos que entregarem uma rota válida dentro do limite de 15% de tolerância, o ranqueamento será feito pela soma de nós expandidos. Vence o algoritmo que for mais veloz (menor expansão).
- O grupo mais eficiente leva 1,0 ponto. Os demais recebem a nota proporcional, utilizando a mesma fórmula do Desafio 1.

3. Métricas de Desempenho

Para avaliar a qualidade técnica e a eficiência computacional da sua solução, o sistema será julgado por duas métricas complementares:

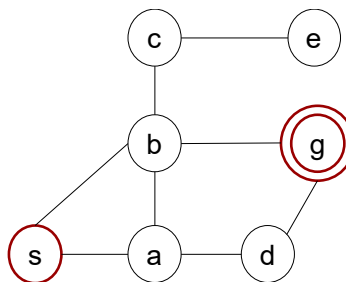
3.1. Distância Total da Rota: o sistema deve calcular a distância total percorrida (soma do custo das arestas, em metros) do caminho encontrado. Para que a sua solução seja considerada ótima, ela deve, obrigatoriamente, encontrar a rota mais curta possível. Algoritmos que encontram caminhos válidos, mas que apresentam uma distância total maior que a rota ótima conhecida, são considerados sub-ótimos. Algoritmos sub-ótimos serão aceitos exclusivamente na avaliação do Desafio 2, desde que a distância excedente não ultrapasse a margem de 15% do caminho ótimo.

3.2. Contagem de Nós Expandidos: o sistema deve calcular o número de nós expandidos. Deve ser adotada a seguinte definição formal de expansão.

Definição: Um nó é considerado *expandido* no exato momento em que ele é retirado da Fronteira (Fila/Pilha/Fila de Prioridade) para que seus vizinhos (filhos) sejam descobertos e avaliados. Note que apenas olhar para os vizinhos de um nó atual e inseri-los na fronteira **não** conta como expansão. A expansão só ocorre quando o nó se torna o "nó atual" da iteração.

Exemplo

Imagine o grafo seguinte, onde *s* é a origem e o destino é *g*.



1. Início:
 - Contador de Expansões = **0**.
 - Fronteira = [**[s]**]
2. Iteração 1: Retiramos **[s]** da Fronteira. Não é o destino. Seus filhos **a** e **b** são gerados e colocados na Fronteira.
 - Fronteira atual: [**[a,s]**, **[b,s]**]
 - Expansões = **1** (Nó processado: **s**).
3. Iteração 2: Retiramos **[a,s]** da Fronteira. Não é o destino. Filhos de **a**: **b**, **d** são colocados na Fronteira.
 - Fronteira atual: [**[b,a,s]**, **[d,a,s]**, **[b,s]**]
 - Expansões = **2** (Nós processados: **s, a**).
4. Iteração 3: Retiramos **[b,a,s]** da Fronteira. Não é o destino. Filhos de **b**: **c**, **g** são colocados na Fronteira.
 - Fronteira atual: [**[c,b,a,s]**, **[g,b,a,s]**, **[d,a,s]**, **[b,s]**]
 - Expansões = **3** (Nós processados: **s, a, b**).
5. Iteração 4: Retiramos **[c,b,a,s]** da Fronteira. Não é o destino. Filho de **c**: **e** é colocado na Fronteira.
 - Fronteira atual: [**[e,c,b,a,s]**, **[g,b,a,s]**, **[d,a,s]**, **[b,s]**]
 - Expansões = **4** (Nós processados: **s, a, b, c**).
6. Iteração 5: Retiramos **[e,c,b,a,s]** da Fronteira. Não é o destino. Filhos de **e**: nenhum filho válido gerado (beco sem saída). O nó é descartado, mas a expansão é contabilizada porque a CPU executou a avaliação.
 - Fronteira atual: [**[g,b,a,s]**, **[d,a,s]**, **[b,s]**]
 - Expansões = **5** (Nós processados: **s, a, b, c, e**).
7. Iteração 6: Retiramos **[g,b,a,s]** da Fronteira. Como **g** é o nó final, a busca termina.
 - Retorna o caminho (invertido): **[s, a, b, g]**
 - O contador não incrementa nesta etapa porque o nó **g** foi apenas testado, não expandido.
8. Resultado Final:
 - Caminho: **[s, a, b, g]**
 - Total de Expansões: **5**

Entrega e Submissão:

- a) O trabalho pode ser desenvolvido em grupo de até três alunos (sem exceções); um único trabalho deve ser entregue pelo grupo.
- b) Os nomes dos integrantes do grupo com seus respectivos números USP devem estar presentes em todos os arquivos-fonte, na forma de comentário.
- c) Todos os arquivos desenvolvidos devem ser entregues em um **único** arquivo de formato ZIP no e-Disciplinas, na respectiva atividade, incluindo a apresentação.

Template de Apresentação (Aprox. 15 minutos)

Slide 1: Capa

- Nome da Disciplina e Título do Trabalho.
- Nome e número USP dos 3 integrantes do grupo.

Slide 2: A Escolha dos Algoritmos

- Qual(is) algoritmo(s) de busca foi(foram) implementado?
- Se usaram heurística, como ela foi modelada?
- Como o grupo resolveu o problema das grandezas físicas (coordenadas polares, distância em metros)?
- Qual era a expectativa teórica de desempenho desse(s) algoritmo(s) antes de rodar os testes?

Slide 3: Estruturas de Dados e Implementação

- Como o grafo foi representado na memória a partir dos arquivos **.csv** (Listas de adjacência, matrizes, dicionários)?
- Qual estrutura de dados foi utilizada para a **Frenteira** (Pilha, Fila, Fila de Prioridade, etc)? Por que essa escolha foi feita?
- Como o código garante que a regra de contagem de "Nós Expandidos" está sendo rigorosamente respeitada?

Slide 4: Desafios Técnicos e Soluções

- Quais foram os maiores obstáculos de implementação? (Ex: lidar com grafos desconexos, estouro de memória, nós sem saída, tempo excessivo de CPU).
- Quais otimizações foram feitas para tornar o código mais rápido?

Slide 5: Resultados Iniciais e Validação Visual

- Apresentação de uma rota de teste escolhida pelo grupo.
- Mostrar o *print* da saída padronizada do terminal (Distância Total, Nós Expandidos, Porcentagens).
- Mostrar o mapa interativo renderizado pelo script **verRota.py**, provando visualmente que o algoritmo faz rotas lógicas e segue a malha viária.

Slide 6: Os Desafios (grupo completo)

- Slide reservado para a dinâmica da aula. *Para garantir que a apresentação flua bem, um estudante do grupo deve deixar o ambiente preparado (terminais abertos, abas do navegador prontas para abrir o HTML gerado).*
- O grupo deve estar pronto para receber as **3 rotas** de teste do professor.
- Execução ao vivo para o Desafio 1 (foco em otimalidade) e para o Desafio 2 (foco em velocidade com tolerância de 15%).
- Anotação dos resultados da "Distância Total" e "Nós Expandidos" na planilha da sala para ambos os desafios.
- Plotagem rápida de pelo menos uma dessas rotas teste usando o **verRota.py** para a sala visualizar o caminho encontrado. Recomenda-se mostrar as rotas (Desafios 1 e 2) de cores distintas.

Slide 7: Conclusão e Trade-offs

- O algoritmo escolhido em cada desafio se provou a melhor opção?
- Como o grupo manipulou o algoritmo no Desafio 2 para abrir mão da otimalidade em troca de velocidade?

- O *trade-off* valeu a pena? A porcentagem de expansões de nós poupadas (-X%) justificou o aumento na distância percorrida (+Y%)?
- Em uma aplicação comercial real (como Waze, Uber ou iFood), qual das duas abordagens o grupo adotaria como padrão e por quê?
- Se o grupo tivesse mais tempo, o que faria de diferente para otimizar ainda mais a estrutura de dados (Fronteira) ou para diminuir ainda mais o número de expansões?